

LOG BOOK

INTELLIGENT TRAFFIC MANAGEMENT SYSTEM DEVELOPMENT LOGBOOK

Project Title: Intelligent Traffic Management System (Android - Kotlin)

Student Name: IAN MIRO

Registration Number: BIT/2024/54702

Institution: MKU

Supervisor: MR MICHAEL NYORO

Duration: 7 Weeks

Week 1: User Interface (UI) Design

Activities Performed

- Installed Android Studio and configured the development environment.
- Created a new Android project using Kotlin.
- Studied Material Design principles for Android applications.
- Designed wireframes for the Login, Registration, Dashboard, Traffic Reports, and Settings screens.
- Created XML layout files using `ConstraintLayout`, `LinearLayout`, `CardView`, and `RecyclerView`.
- Added buttons, text fields, image views, icons, and navigation components.

Tools Used

- Android Studio
- XML Layout Editor
- Material Design Components

Challenges Encountered

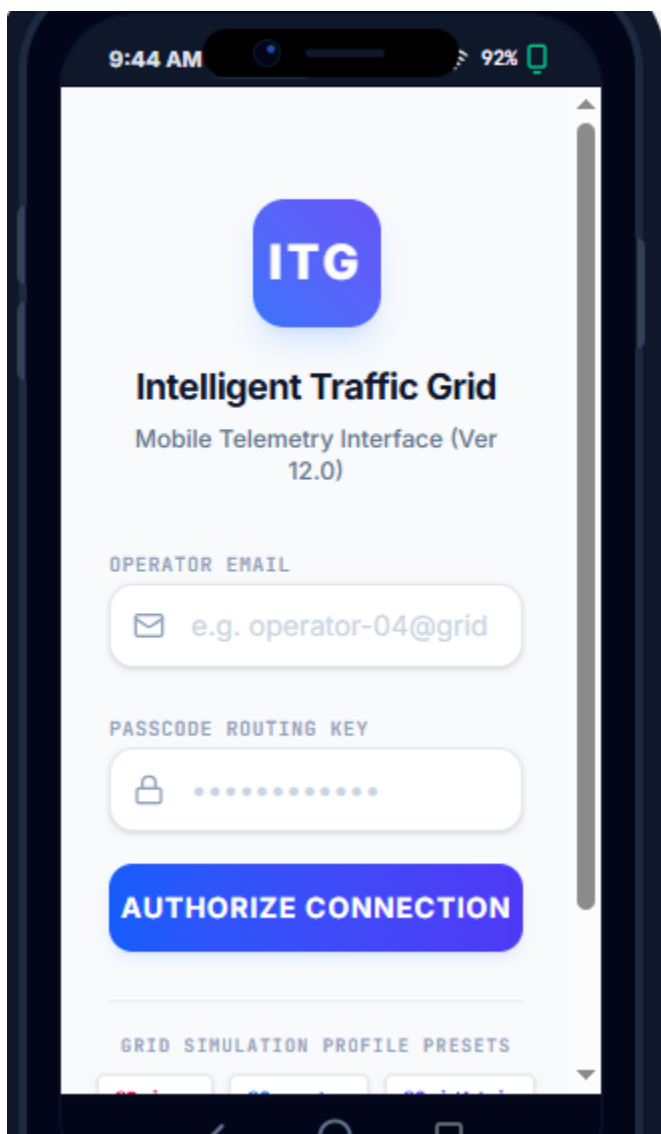
- Aligning UI components properly across different screen sizes.
- Understanding nested layouts and constraints.

Solutions

- Used `ConstraintLayout` for responsive design.
- Tested layouts using different emulator screen sizes.

What I Learned

- Android UI is designed using XML layouts.
- Material Design improves consistency and user experience.
- `ConstraintLayout` provides flexible and responsive positioning.
- Good UI design enhances usability and accessibility.
- The importance of spacing, colors, and typography in mobile applications.



Week 2: Navigation

Activities Performed

- Designed application flow.
- Connected Login to Dashboard.
- Connected Registration to Login.
- Implemented Bottom Navigation.
- Implemented Navigation Drawer.
- Added intents for moving between activities.

Tools Used

- Navigation Component
- Intents
- Android Studio

Challenges Encountered

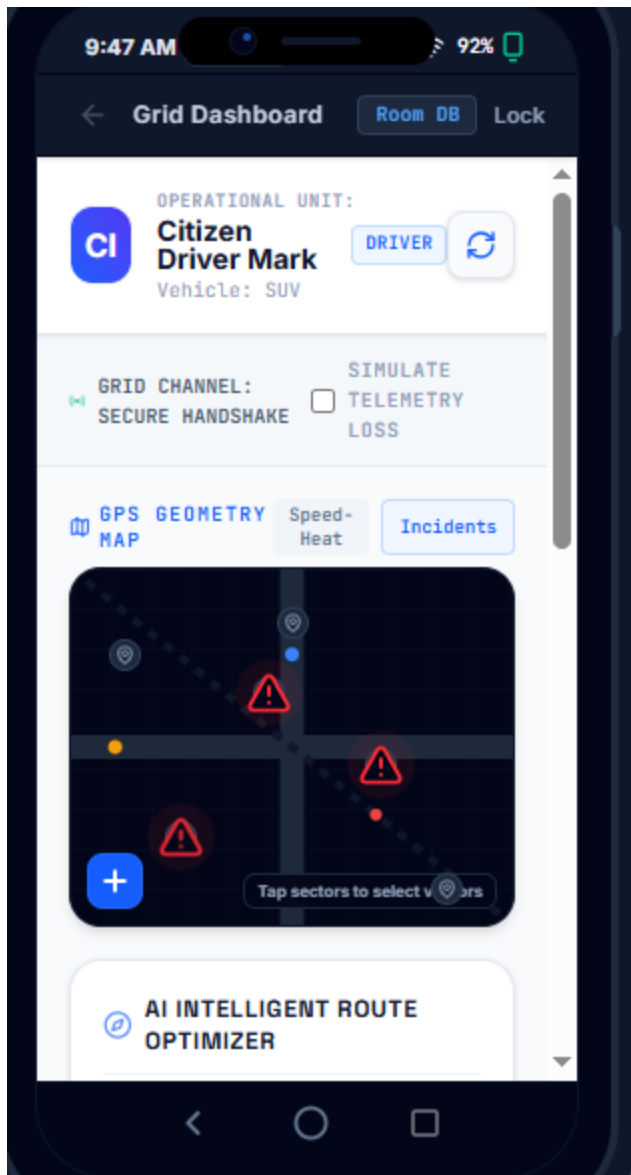
- Passing data between activities.
- Managing the activity back stack.

Solutions

- Used Intent Extras.
- Implemented proper navigation hierarchy.

What I Learned

- Activities represent different application screens.
- Intents are used for communication between activities.
- Navigation Drawer improves accessibility.
- Bottom Navigation simplifies movement between major screens.
- Navigation Component simplifies screen transitions.



Week 3: Event Handling

Activities Performed

- Added button click listeners.
- Implemented search functionality.
- Implemented RecyclerView item click events.
- Added form validation.
- Added swipe refresh functionality.

Tools Used

- Kotlin Event Listeners
- RecyclerView
- SearchView

Challenges Encountered

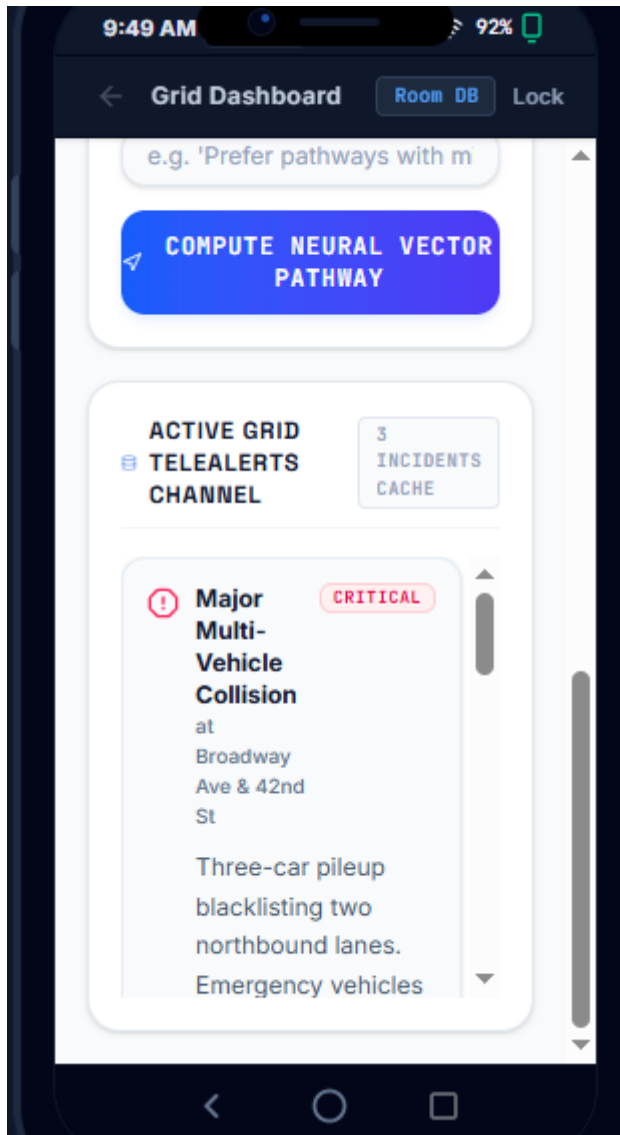
- Handling multiple click events.
- Preventing application crashes from invalid input.

Solutions

- Used Kotlin lambda expressions.
- Implemented input validation before processing.

What I Learned

- Event handling makes applications interactive.
- Every button requires an event listener.
- Form validation improves data quality.
- RecyclerView supports user interaction through item clicks.
- SearchView enables real-time filtering.



Week 4: Local Data Storage

Activities Performed

- Implemented Room Database.
- Created User Entity.
- Created Incident Entity.
- Created DAO interfaces.
- Stored registered users locally.
- Stored reported traffic incidents locally.

Tools Used

- Room Database
- SQLite
- Kotlin

Challenges Encountered

- Understanding Entity relationships.
- Creating SQL queries using annotations.

Solutions

- Studied Room Database architecture.
- Used Room annotations correctly.

What I Learned

- Room Database simplifies SQLite implementation.
- Entities represent database tables.
- DAO interfaces perform CRUD operations.
- Room supports compile-time SQL validation.
- Local storage enables offline functionality.



Week 5: Data Retrieval

Activities Performed

- Retrieved user information from Room Database.
- Displayed stored incidents in RecyclerView.
- Added search functionality.
- Observed LiveData changes.

Tools Used

- LiveData
- ViewModel
- RecyclerView

Challenges Encountered

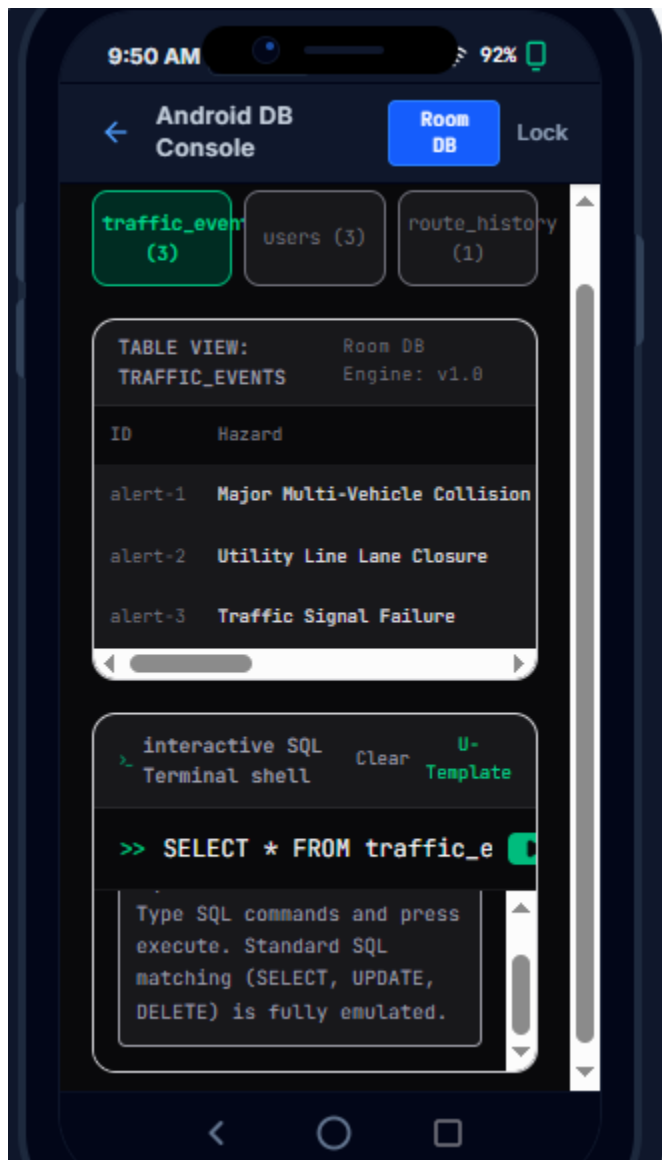
- Updating RecyclerView after inserting data.
- Managing asynchronous operations.

Solutions

- Used LiveData observers.
- Used ViewModel for lifecycle management.

What I Learned

- LiveData automatically updates the UI.
- ViewModel preserves data during configuration changes.
- RecyclerView efficiently displays dynamic lists.
- Data retrieval should occur asynchronously.



Week 6: Networking

Activities Performed

- Learned REST API concepts.
- Configured Retrofit.
- Connected the application to a remote server.
- Implemented GET and POST requests.
- Configured Gson Converter.

Tools Used

- Retrofit
- Gson
- Coroutines

Challenges Encountered

- API connection failures.
- JSON parsing errors.

Solutions

- Verified endpoint URLs.
- Used correct data models.

What I Learned

- REST APIs exchange information between client and server.
 - Retrofit simplifies HTTP communication.
 - Coroutines simplify background operations.
 - JSON is the standard API response format.
 - Network operations should not run on the main thread.
-

Week 7: Event Handler

Activities Performed

- Retrieved traffic data from the server.
- Displayed incidents in RecyclerView.
- Uploaded traffic reports to the server.
- Consumed weather and traffic APIs.

Tools Used

- Retrofit
- Gson
- RecyclerView

Challenges Encountered

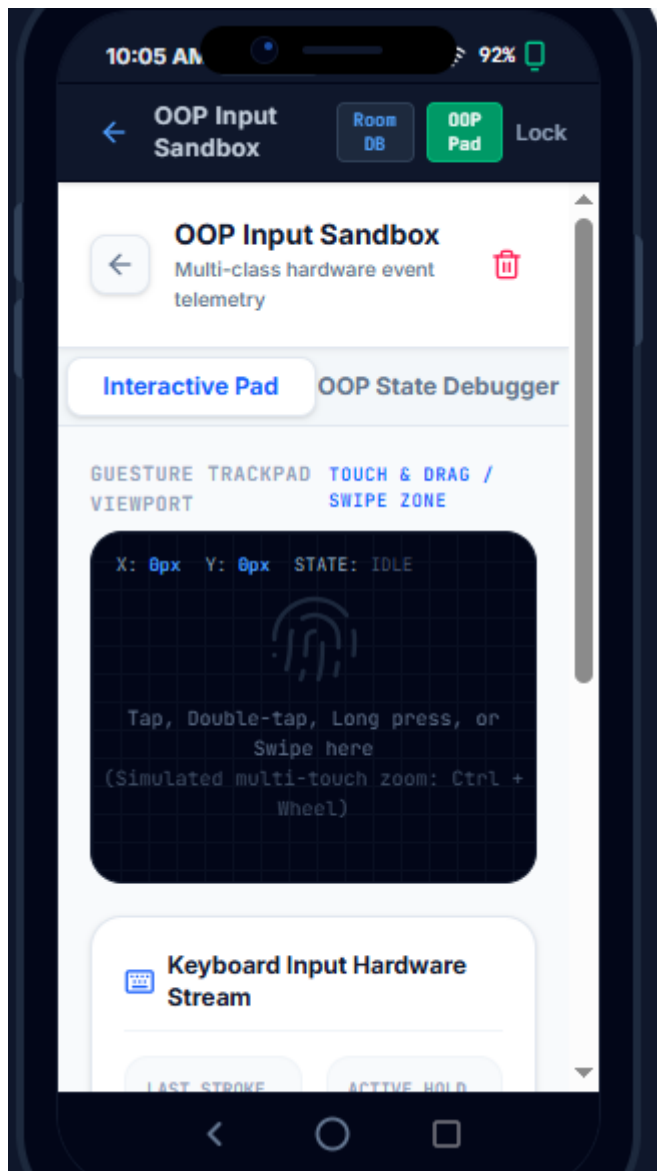
- API authentication.
- Parsing nested JSON responses.

Solutions

- Created Kotlin Data Classes.
- Implemented response checking.

What I Learned

- APIs enable communication between applications.
- Retrofit converts JSON into Kotlin objects automatically.
- Data classes simplify JSON parsing.
- API consumption provides live traffic information.
- Error handling is important during network communication.





Reflection

The development of the Intelligent Traffic Management System enhanced my Android development skills and strengthened my understanding of software engineering concepts such as user interface design, navigation, event handling, database management, networking, API integration, and exception handling. The project also improved my problem-solving abilities and demonstrated the importance of writing maintainable, scalable, and user-friendly mobile applications.